

Lattice Tooling: Omnidirectional Numbering and Nearest-Site Search

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Lattice Tooling: Omnidirectional Numbering and Nearest-Site Search	1
1.1 Overview	1
1.2 Omnidirectional Close Packing and Frequency Shells	1
1.2.1 Reachability is enumerative, not congruence-based	2
1.2.2 Site/index mapping	2
1.3 Fast Nearest-Site Queries	2
1.3.1 Exact lattice distances	2
1.3.2 Truncation bound	2
1.3.3 Shell-sweep nearest	3
1.4 API Summary	3
1.5 Verification	3

1 Lattice Tooling: Omnidirectional Numbering and Nearest-Site Search

1.1 Overview

Sections 2 and 3 develop the Quadray/IVM framework analytically; this section documents the computational tooling that operates directly on the close-packed lattice. Two modules extend the analytical core without touching it:

- `omni_numbering` — omnidirectional close-packing numbering: vectorized enumeration of the IVM shell sequence, cumulative counts, and bidirectional site/index mappings.
- `lattice_search` — fast nearest-lattice-point queries: `nearest(site, R, k)` and `within_radius(site, R)` built on a precomputed ball index, entirely in NumPy (no scipy).

Both modules operate on canonical quadray integer 4-tuples (non-negative components, at least one zero — the projective normalization of Section 3), and both rely on the same verified shell invariants.

1.2 Omnidirectional Close Packing and Frequency Shells

In the isotropic vector matrix (IVM), equal spheres pack omnidirectionally — closest packing. Starting from a central sphere, the packing builds up in consecutive **frequency shells**: the shell of frequency $k \in \mathbb{Z}_{\geq 0}$ sits k radius-length increments outward of the center, the same four-dimensional buildup language synergetics uses for the growing vector equilibrium. Writing N_k for the population of shell k , the lone center is the whole of shell 0 ($N_0 = 1$), and every shell with $k \geq 1$ carries exactly

$$N_k = 10 k^2 + 2 \quad (1)$$

sphere centers. The first shell — the 12 permutations of (2, 1, 1, 0) — is the cuboctahedron (vector equilibrium) of the “twelve around one” motif; each subsequent shell is the next layer of the omnidirectional buildup. The cumulative count through frequency k , written C_k , is the centered-cuboctahedral closed form

$$C_k = 1 + \sum_{j=1}^k (10 j^2 + 2) = 1 + 2k + \frac{10 k (k + 1) (2k + 1)}{6}. \quad (2)$$

The functions `shell_count(k)` and `cumulative_count(k)` evaluate Eqs. (1) and (2) (scalar or array input), and the test suite cross-validates them against direct summation.

1.2.1 Reachability is enumerative, not congruence-based

A normalized integer quadray q with $\sum_i q_i \equiv 0 \pmod{4}$ is a candidate IVM point, but not every candidate is reachable from the origin by the 12 neighbor moves. The module therefore builds the site set **layer by layer**: each new shell is produced at once by adding all 12 moves to the previous frontier, re-normalizing rows, and removing sites already seen (packed-key membership tests). The result is cached per depth: `sites_through_shell(k)` returns the `(C_k, 4)` integer array in canonical order — the center first, then each shell in lexicographic (a, b, c, d) order — and `generate_shell(k)` returns one shell. An independent breadth-first reference over `itertools.permutations` confirms exact agreement through frequency 8, including the counts of Eq. (1) on every shell.

1.2.2 Site/index mapping

`site_index(site, max_shell)` returns the position of a site in that canonical enumeration (or -1 if absent within the depth), and `site_at_index(index, max_shell)` is its inverse. Lookups accept any projective representative — the input is translated by $-(k, k, k, k)$ before matching — so (2, 1, 1, 0) and (3, 2, 2, 1) map to the same index, while (0, 1, 1, 2), a different permutation and hence a different sphere, maps elsewhere. Component overflow is guarded: inputs whose normalized components reach the 16-bit packed-key width (2^{16} , the per-component field of the int64 keys) return -1 rather than wrapping.

Executable check of the bookends:

```

1 from quadmath.lattice.omni_numbering import (
2     shell_count, cumulative_count, sites_through_shell,
3     site_index, site_at_index,
4 )
5
6 assert shell_count(1) == 12 and shell_count(2) == 42      # 10k^2 + 2
7 assert cumulative_count(2) == 55                          # 1 + 12 + 42
8 assert site_index(site_at_index(12, 3), 3) == 12          # roundtrip
9 assert site_index((2, 0, 0, 0), 6) == -1                  # unreachable site

```

1.3 Fast Nearest-Site Queries

1.3.1 Exact lattice distances

Under `quadray.DEFAULT_EMBEDDING` the four embedding columns C_j satisfy $C_i \cdot C_j = 4 \delta_{ij} - 1$ at unit scale, so the Euclidean squared distance between integer 4-vectors p, q is the exact integer

$$d^2(p, q) = 4 \sum_j \delta_j^2 - \left(\sum_j \delta_j \right)^2, \quad \delta = p - q, \quad (3)$$

implemented in `squared_distance` and cross-validated in the tests against direct embedding coordinates. Because the identity is exact on integers, ranking and filtering never suffer floating-point boundary errors: a site is inside the query radius $R > 0$ iff $d^2 \leq R^2$ evaluated exactly (in float64 the same algebra is exact for the magnitudes involved here).

1.3.2 Truncation bound

Every site s on frequency shell g satisfies $d^2(\mathbf{0}, s) \geq 8g$ — verified by exhaustive enumeration through frequency 8 in the test suite — and the shell maximum is exactly $8g^2$. Combining this invariant with the triangle inequality gives the depth at which a query can stop:

$$g \leq \frac{(\|c\| + R)^2}{8}, \quad (4)$$

for a query center c and radius $R > 0$, where $\|c\|$ is the Euclidean norm of the embedded center: any site within R of c must live on a shell at most this deep. `within_radius(site, R)` therefore enumerates exactly through $\lfloor (\|c\| + R)^2/8 \rfloor + 1$ shells, filters with Eq. (3), and returns all hits sorted by squared distance with lexicographic (a, b, c, d) tie-breaking. Requests whose required depth exceeds the precomputed index depth (`MAX_SHELL = 32`, about 1.1×10^5 sites — $C_{32} = 114465$ by Eq. (2)) raise `ValueError` rather than silently truncating.

1.3.3 Shell-sweep nearest

`nearest(site, R, k)` sweeps shells outward, ranking each shell's squared distances with `numpy.argsort`; once k candidates are in hand, the invariant behind Eq. (4) supplies an early-stop test — no unvisited shell can contain a closer site: a site at depth $g+1$ satisfies $d^2(\mathbf{0}, s) \geq 8(g+1)$, so the sweep stops once $8(g+1) > (\sqrt{d_{(k)}^2} + \|c\|)^2$, with $d_{(k)}^2$ the current k -th best squared distance. The sweep never under-enumerates (the bound is conservative; ties at the boundary are kept), and answers always match the brute-force reference over the same region, as the tests assert on fixed-seed random queries.

```
1 import numpy as np
2 from quadmath.lattice.lattice_search import nearest, within_radius
3
4 sites, d2 = nearest((0.4, -0.3, 0.9, 0.1), R=2.0, k=5) # 5 nearest centers
5 ball, ball_d2 = within_radius((2, 1, 1, 0), R=4.0) # everything within 4
6 assert np.all(np.diff(ball_d2) >= 0) # ascending distances
```

1.4 API Summary

Function	Purpose
<code>omni_numbering.shell_count(k)</code>	Shell population, Eq. (1)
<code>omni_numbering.cumulative_count(k)</code>	Cumulative count through shell k , Eq. (2)
<code>omni_numbering.generate_shell(k)</code>	Sites of one frequency shell (lexicographic)
<code>omni_numbering.sites_through_shell(k)</code>	Whole enumeration (C_k , 4) in canonical order
<code>omni_numbering.site_index / site_at_index</code>	Bidirectional site/index mapping with projective normalization
<code>lattice_search.squared_distance</code>	Exact squared distances via Eq. (3)
<code>lattice_search.within_radius(site, R)</code>	All sites within radius, exact and sorted
<code>lattice_search.nearest(site, R, k)</code>	k nearest sites within radius, shell-sweep with early stop

1.5 Verification

Both modules ship with 100% test coverage (branch coverage included): the enumeration is checked against an independent breadth-first reference through frequency 8, distance identities against direct embedding computations, and query results against brute-force filtering on fixed-seed random centers. Run:

```
1 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run coverage run -m pytest -q
2 uv run coverage report
```